

a line, which is represented by the dollar sign (\$). For example:

```
(rx "end" line-end)
"end$"

(s-matches-p (rx "end" line-end)
  "The end.")
nil

(s-matches-p (rx "end" line-end)
  "The end")
t
```

A digit can be represented by using either *digit*, *numeric* or *num* constructs. A couple of examples are shown below:

```
(rx digit)
"[:digit:]"

(rx numeric)
"[:digit:]"

(rx num)
"[:digit:]"

(s-matches-p (rx num)
  "1234")
t

(s-matches-p (rx num)
  "abcd")
nil
```

A control character is a non-printing character and you can use either *control* or *cntrl* constructs to generate the regular expression for the same. A few examples are given below:

```
(rx control)
"[:cntrl:]"

(rx cntrl)
"[:cntrl:]"

(s-matches-p (rx control)
  "\0")
t

(s-matches-p (rx control)
  "abc")
nil
```

A hexadecimal digit can be matched by using either *hex-digit*, *hex*, or *xdigit* rx-constituents as illustrated below:

```
(rx hex-digit)
"[:xdigit:]"

(rx hex)
"[:xdigit:]"

(rx xdigit)
"[:xdigit:]"

(s-matches-p (rx digit)
  "1234")
t

(s-matches-p (rx digit)
  "abcd")
nil
```

You can match for lower case characters using either *lower* or *lower-case* constructs. Similarly, you use *upper* or *upper-case* constructs to match for upper case letters. A few examples are shown below:

```
(rx lower)
"[:lower:]"

(rx lower-case)
"[:lower:]"

(rx upper)
"[:upper:]"

(rx upper-case)
"[:upper:]"

(s-matches-p (rx lower)
  "abc")
t

(s-matches-p (rx lower-case)
  ";")
nil

(s-matches-p (rx upper)
  "ABC")
t

(s-matches-p (rx upper-case)
  ";")
nil
```

If you have escaped characters in your input text, you will need to use either *regexp-quote* or *eval* on the input before being able to apply the regular expression to match on the input string. An example is given below:

```
(eval FORM) ;; Syntax

(setq input "\"Hello, world!\")

(not (s-matches-p input input))
nil

(s-matches-p (regexp-quote input) input)
t

(s-matches-p (rx (eval input)) input)
t
```

The rx.el library also provides support for non-ascii characters, such as multi-byte and accented characters. You can match for the same using the non-ascii construct as shown below:

```
(rx nonascii)
"[:nonascii:]"

(s-matches-p (rx nonascii)
  "ABC")
nil

(s-matches-p (rx nonascii)
  "È")
t
```

The *alpha-numeric* rx-constituent can be used to match for both alphabets and numerals. A couple of examples are provided below for reference:

```
(rx alphanumeric)
"[:alnum:]"

(s-matches-p (rx alphanumeric)
  "abc123")
t

(s-matches-p (rx alphanumeric)
  ";")
nil
```